Classical Control Design Project Part 2

Title: Control of Flexible Structures (Controller)
Authors: Blayze Ashhurst, Charles Langer, Daniel Im, Jaron Tsao, Theodore Hays
Date: December 12, 2025

Introduction

The goal of the project is to use the system model developed in the first part of the design project to design a controller capable of rotating the disks as seen in *Figure (1)*, to the required angle.

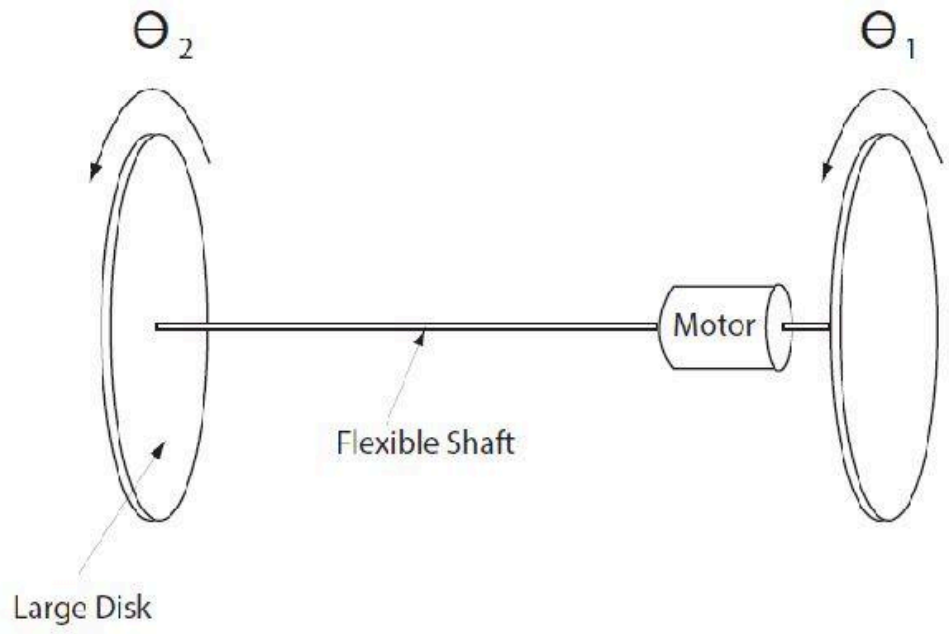


Figure 1: Schematic of a flexible structure, consisting of two large disks connected by a flexible shaft. On one side, a DC motor is used to control the position of the right disk directly Θ_1*. Due to the system's flexibility, the response can be delayed. (1)*

Given two transfer functions, equation *1* represents the position of disk 1 and equation *2* represents the position of disk 2.

$$H_1(s) = \frac{\theta_1}{u} = \frac{21.903s^2 + 1.3465s + 84.2}{s^4 + 0.512s^2 + 6.937s^2 + 2.44s + 0.318} \quad (1)$$

$$H_2(s) = \frac{\theta_2}{u} = \frac{3.588s^2 - 1.186s + 84.2}{s^4 + 0.512s^3 + 6.937s^2 + 2.44s + 0.318} \quad (2)$$

Where the units of the input, *u,* are in volts and the units of the output angles, θ_i, are in degrees.

Objectives

The following are the objectives of this lab:

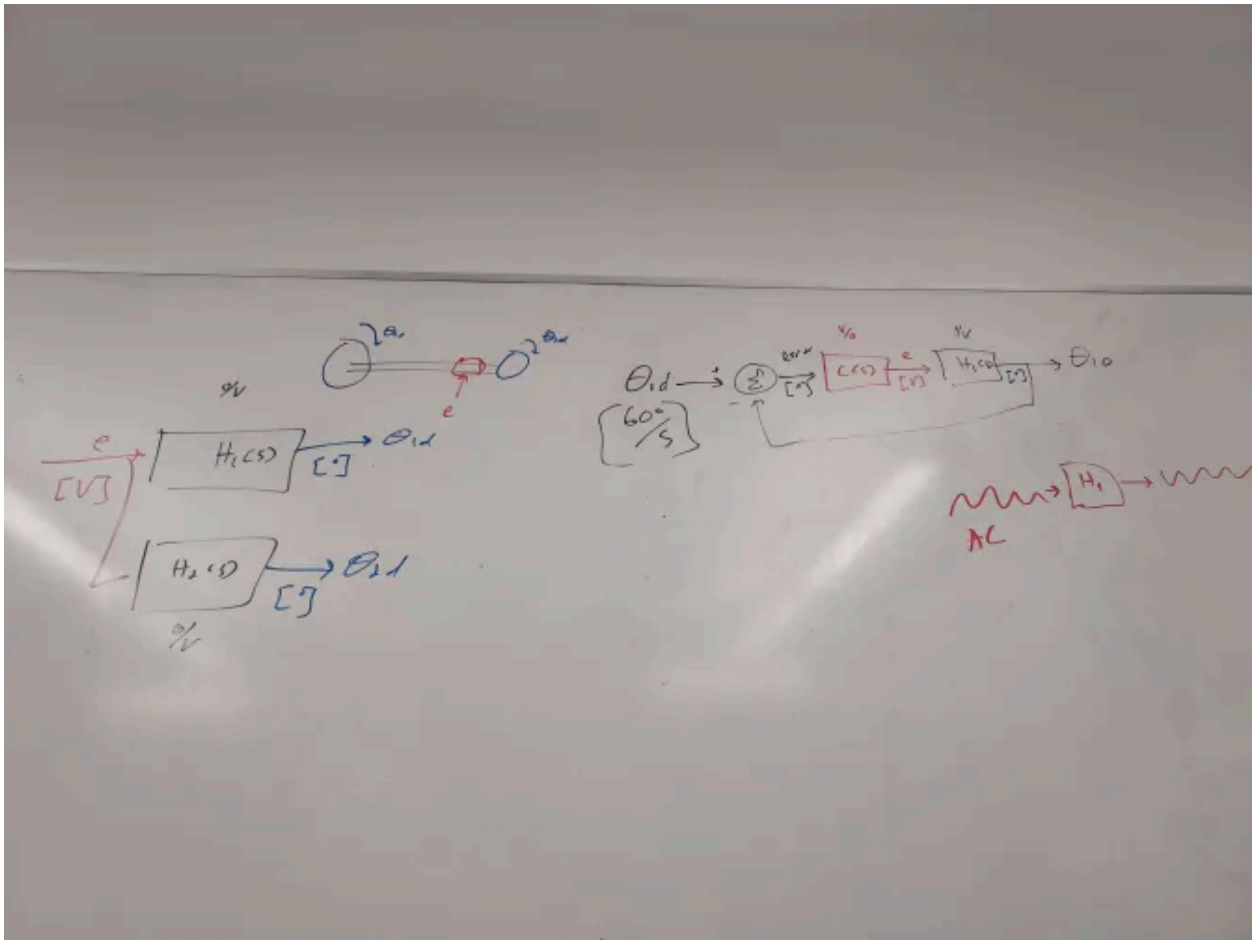
a) Design a controller (PID, root-locus, or lead/lag) such that the system moves from 0° to 60°.
b) Only disk 1 can be in a closed loop.
c) The settling time is less than 15 seconds
d) Signal at controller output/motor input must be less than 5V
e) The angle differences between the two disks must be kept below 10 degrees.
f) Steady state error is reduced to as close to 0 as possible, perhaps within 5 degrees of the desired angle.
g) Draw the Root Locus for the system and create plots that compare the output motor input vs. time and the difference in the angles.

Assumptions

1. Both angles were independently modeled
2. The Plant Transfer function is valid
3. Initial conditions start at rest (0 degrees)
4. Noise and other resistances are negligible

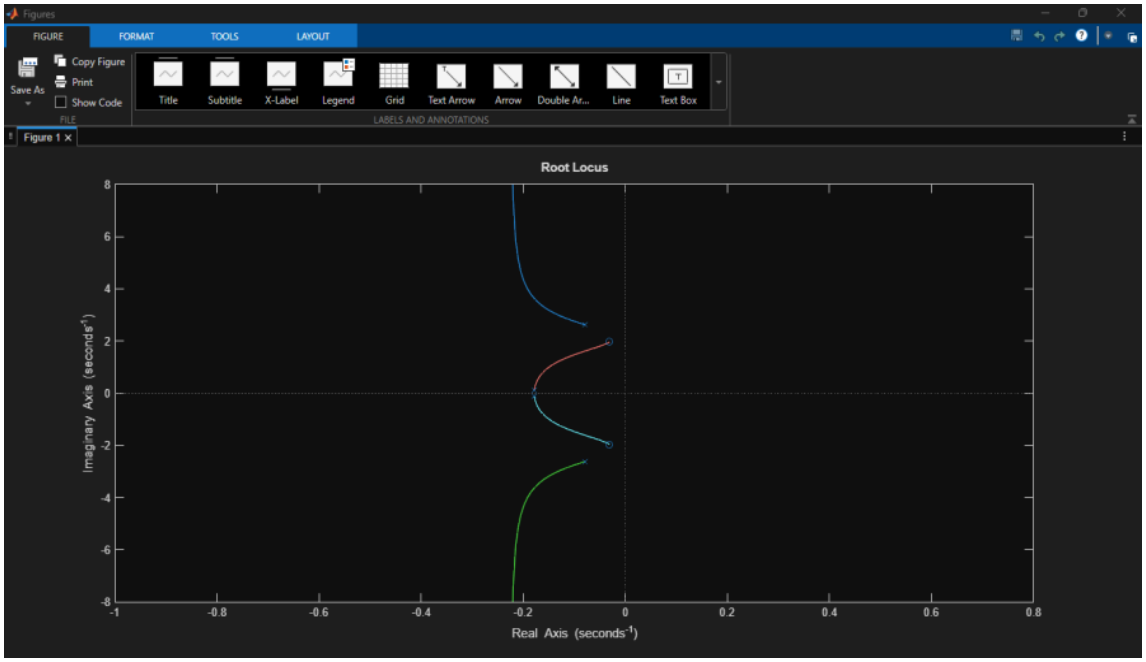
Methods

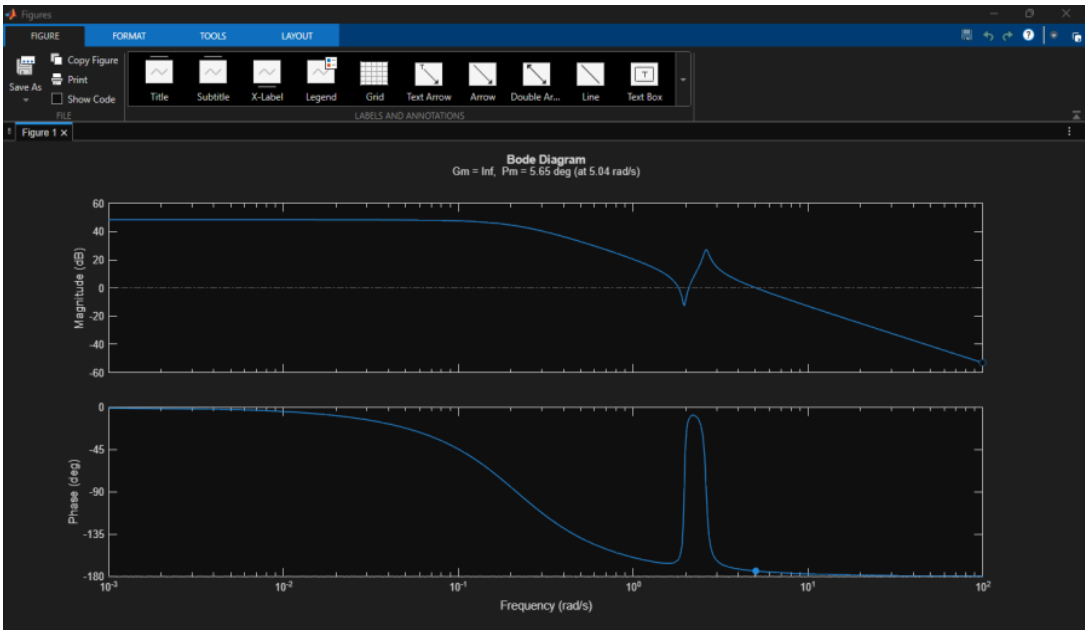
Before designing a controller for the overall system, it was essential to read through the documentation and determine how the closed-loop system would control each disk.



After reading through the documentation and consulting peers, it was determined the system would consist of a controller and disk θ_1 in a closed-loop system with unity gain feedback. To control disk θ_2 simultaneously, the output signal from the controller is fed from a pickoff point into the θ_2 transfer function. Θ_2 has no affect on the closed-loop function between the input and θ_1.

With the overall system configuration understood, the next step of our group was to determine a controller capable of stabilizing the transfer function of disk θ_1 in closed loop. As a group, we decided to focus on optimizing the controller for θ_1 first, then focus on altering it for θ_2. The first attempt we tried was to create a Bode plot and root locus of θ_1, the model for disk 1.

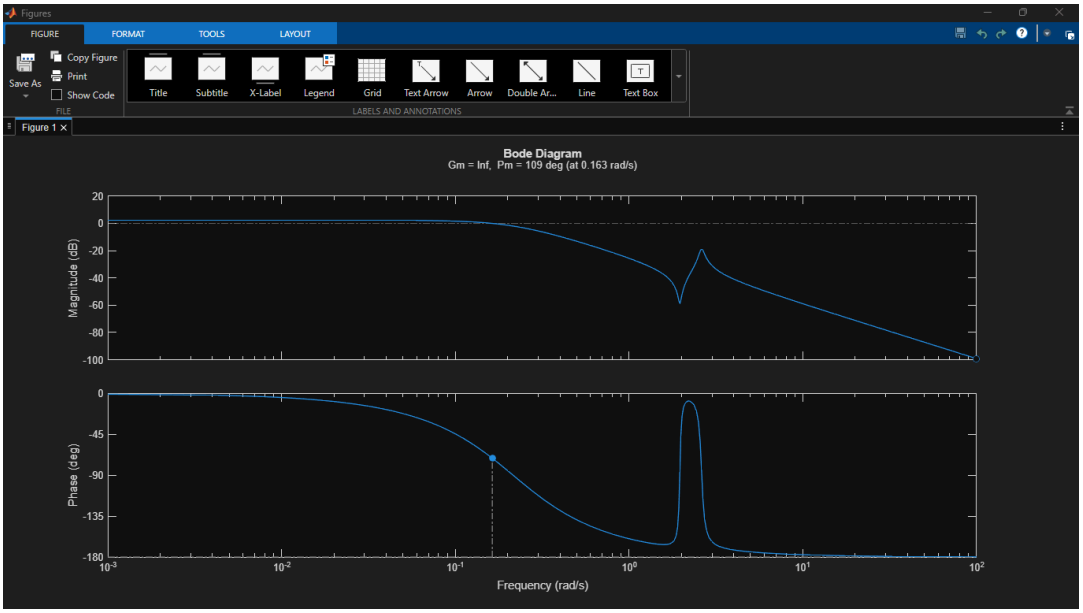




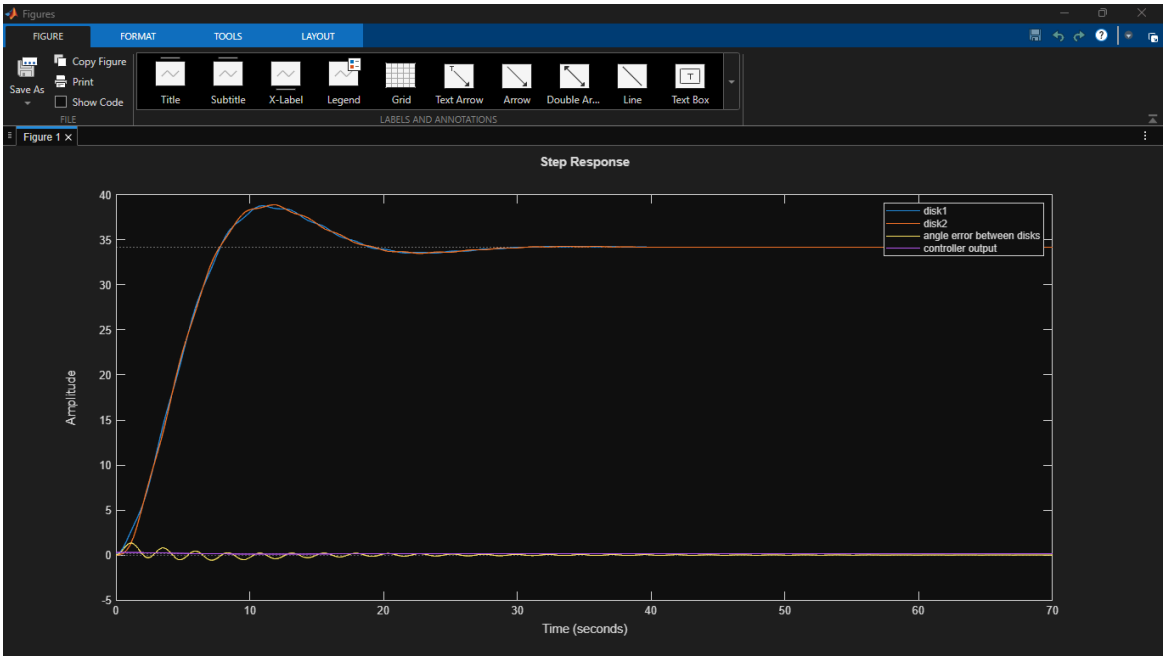
Our group initially planned to use the ultimate sensitivity method of generating PID coefficients for a controller, but according to the root locus plot, no k value makes the poles cross the jw axis. This means that no proportional value alone would make the system oscillate. Additionally, we noticed that with our original PID controller, we did not have the sustained oscillation to use Ziegler-Nichols.

Given the complexity of the root locus plot, we decided to look at the open-loop Bode plot for the controller design. The margin function in MATLAB indicated that the phase margin of the θ_1

transfer function is very small, about 5 degrees. By decreasing the gain with a proportional controller, it is possible to move the 0dB intercept frequency back and improve the phase margin, as the phase seems to rise at lower frequencies before the poles start to take effect.

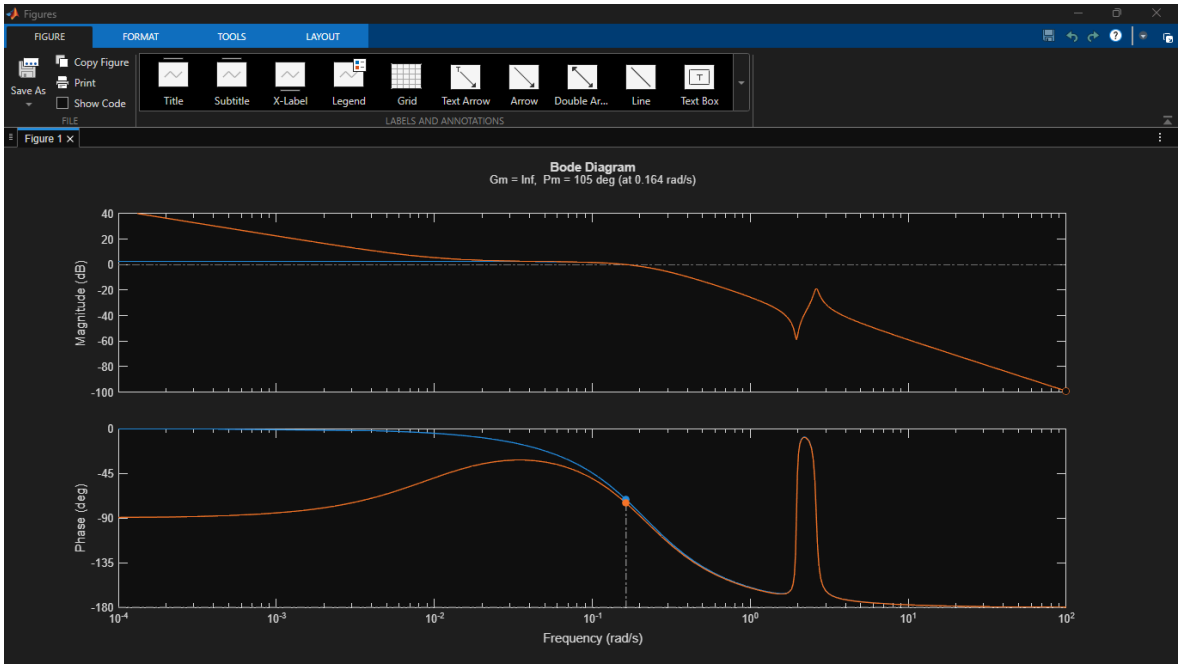


It was found experimentally that a value of .005 for the proportion controller is able to get the phase margin to around 100 degrees. This comes at the sacrifice that the crossover frequency is lowered to about .163 radians per second. The amplitude of the system at lower frequencies is also about .005, which would increase steady state error.



The proportional controller results in both θ_1 and θ_2, the transfer function for each disk, closely following each other when the system is given a step input with magnitude 60. The error between both disks is well beneath the 10-degree maximum, and the controller output is well under 5 volts. However, the transient time and steady state error do not meet the requirements. We decided that further refinement of the controller was necessary.

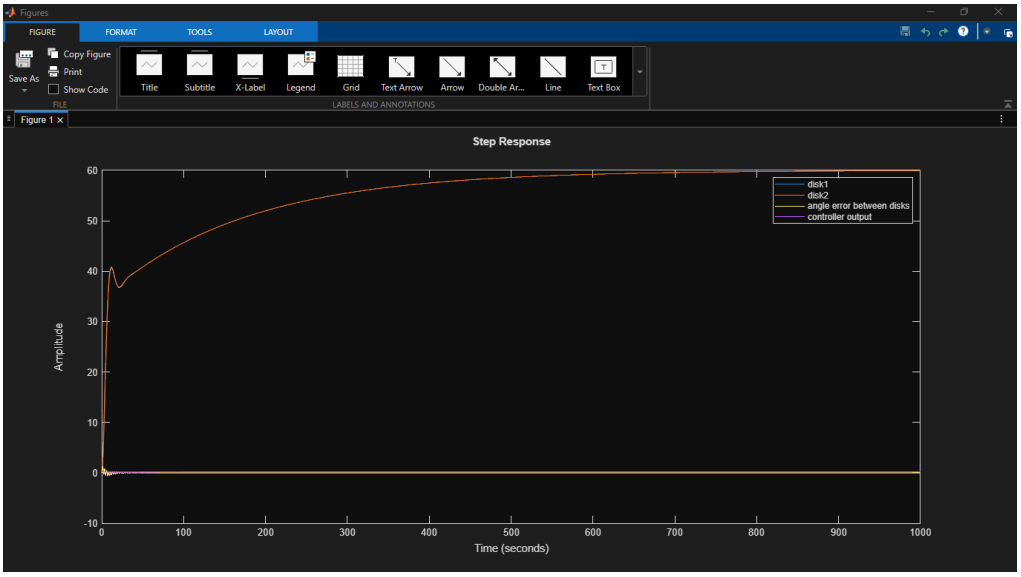
We decided to first start by decreasing the steady state error with a PI controller. A lag controller was considered, but ultimately dropped as a PI controller could be part of a potential PID controller for further refinement of the transient response. We were aware that the integral action could slow down the system with some overshoot.



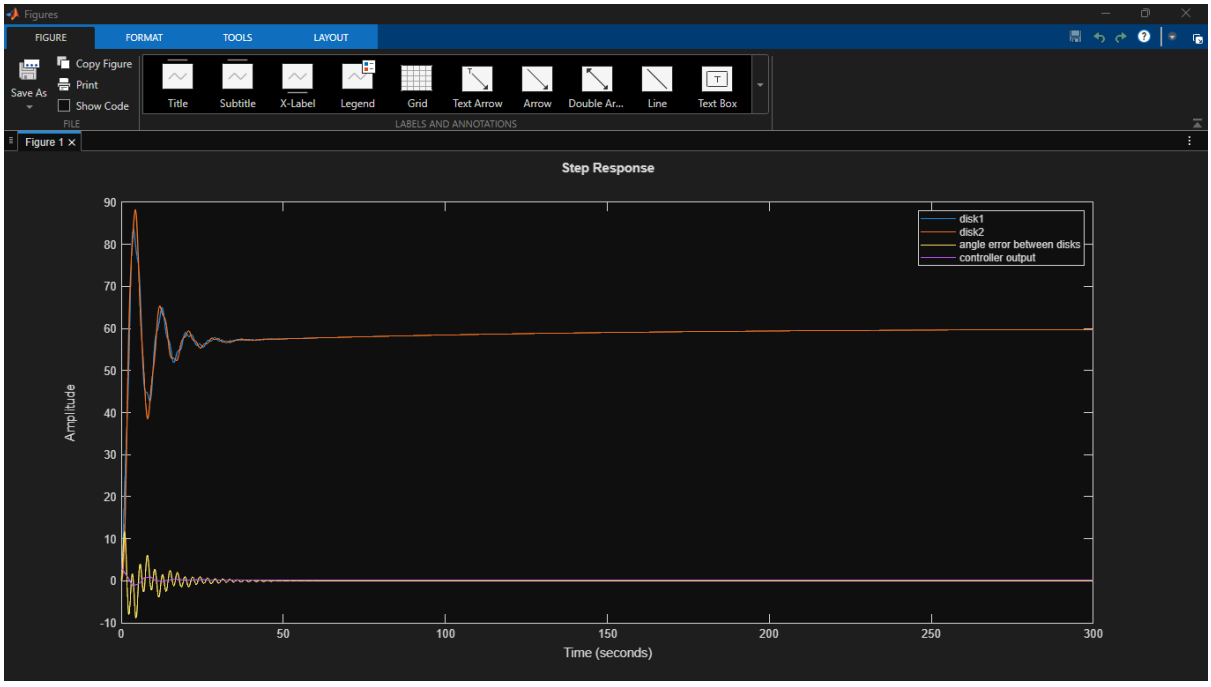
Blue represents the θ_1 transfer function with a proportional controller of .005, orange represents the θ_1 transfer function with a PI controller of the form seen in equation *(3.1)*.

$$C(s) = \frac{0.005(100s + 1)}{100s} \tag{3.1}$$

This controller was designed such that the zero and pole cancel out at 1/100 rad/s, which is low enough as to have marginal effects on phase margin and transient response of the system. As seen, the settling time has increased drastically compared to the PI controller we had implemented previously.



This PI controller proved very effective at removing the steady state error from the system; however, the transient response still needs refinement. The percent overshoot was still too much, and we thought it could have been an issue with the Integral part of this controller, which we shall later refine.

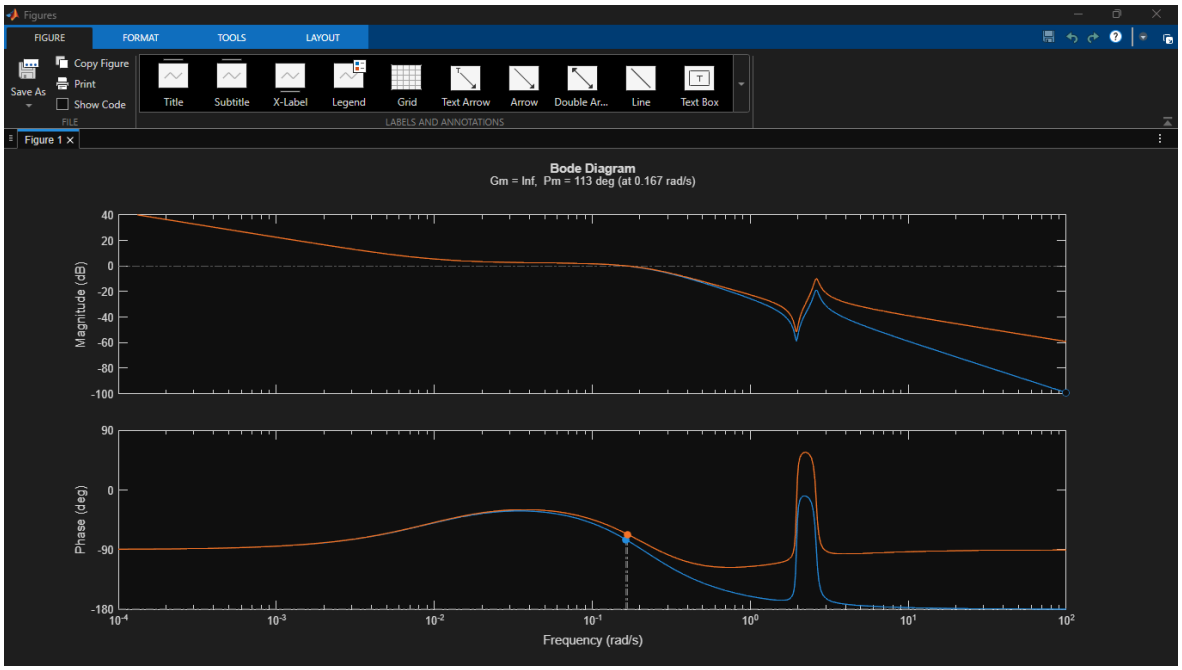


We increased the coefficient on the PI controller from .005 to .05. This reduced settling time, but it is still too large, greater than 30 seconds. Significant ringing and angle difference between the disks also become issues.

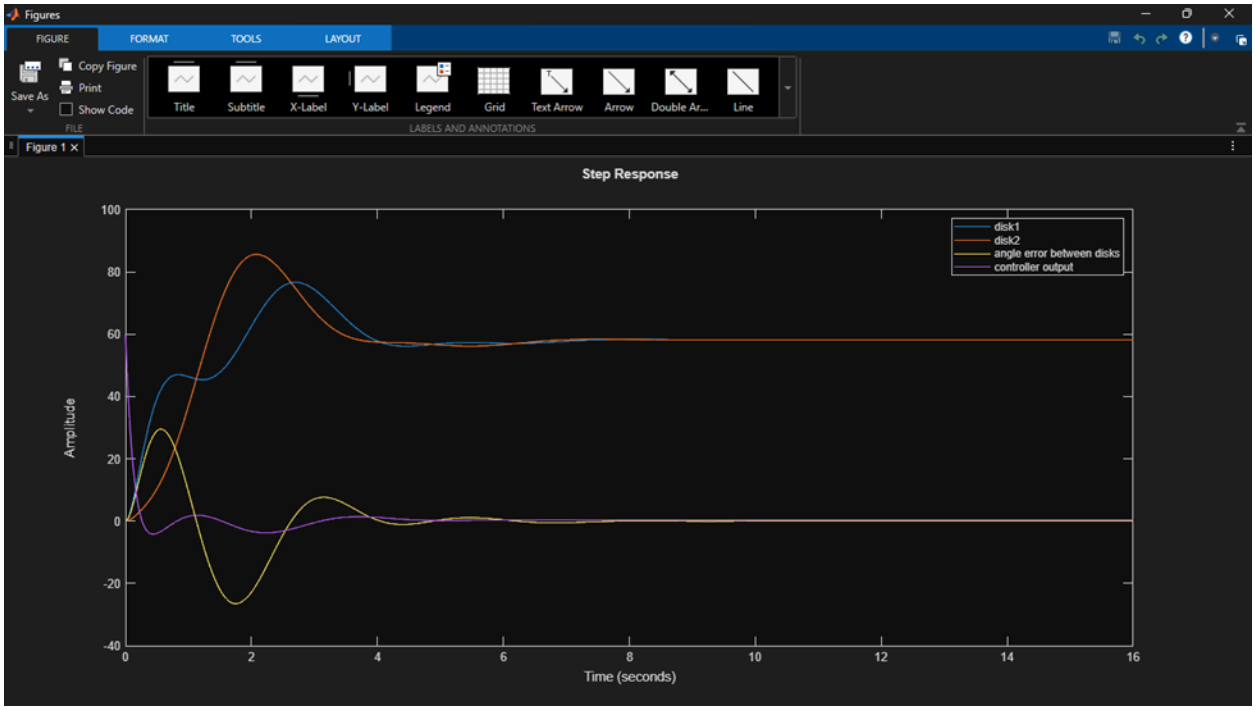
To help reduce ringing and increase the crossover frequency, we decided to add a PD controller in conjunction with the PI, see equation *(3.2)*.

$$C(s) = \frac{0.005*(100s+1)(s+1)}{100s} \tag{3.2}$$

A zero was added at 1 rad/s because in prior plots of θ_1, 1 rad/s is roughly where the poles in the system cause the phase to fall to about -135 degrees, or 45 degrees below -90. This seems to be the ideal point where the 45-degree phase contribution from the zero at its corner frequency should take effect.



Blue is a system with only a PI compensator, and orange is a system with a PI and PD controller. The PI controller added around 90 degrees of phase after the crossover frequency, as hoped, which should allow the gain to be increased with minimum reduction in phase margin.



The transient response after taking the PI and PD controller after increasing the gain to 0.1, see equation *(3.3)*. Blue plot is θ_1 output and orange is Θ_2 output. All plots assume a step input with a magnitude of 60.

$$C(s) = \frac{0.1(100s+1)(s+1)}{100s} \tag{3.3}$$

Transient time is reduced to less than 15 seconds, which is within acceptable tolerances. However, the error signal in yellow indicates that the error between the two disks is exceeding 10 degrees. For the final refinements, our group decided to multiply the controllers together to get a PID controller of the form seen in equation *(3.4)*.

$$C(s) = \frac{s}{10} + 0.101 + \frac{0.001}{s} \tag{3.4}$$

It was observed that increasing the derivative constant introduces ringing in the response, while increasing the integral gain generally helps the system reach a steady state faster. Using this knowledge, the controller coefficients for the PID were refined further, as seen below in equation (*3.5*).

$$C(s) = \frac{s}{50} + 0.01 + \frac{0.0014}{s} \tag{3.5}$$

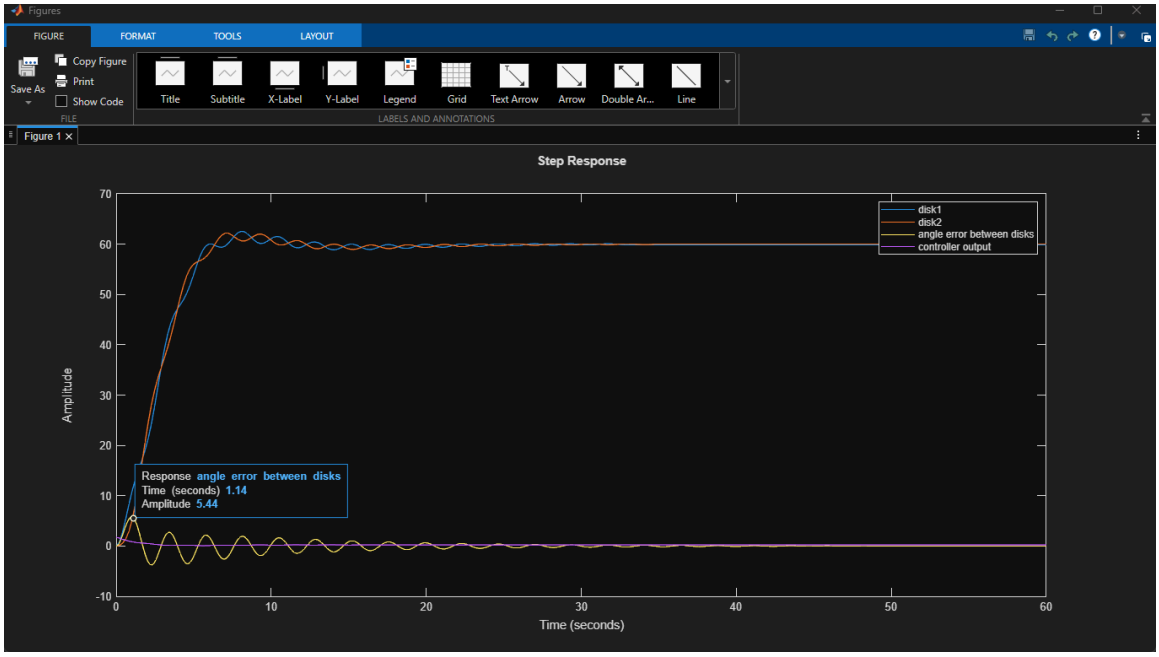
It was observed that some problems were occurring with the plot of the controller output. Matlab was giving continuous errors that the system at the controller output was non-causal and incompatible with the step plot function. On the recommendation of the TA, a filter was added to the derivative portion of the PID controller. This additional pole was selected to have the smallest effect possible on the overall system while increasing the order of the denominator in the controller, thus allowing the controller to become casual and plottable with Matlab's stepplot function. See equation *(4)* for the final controller.

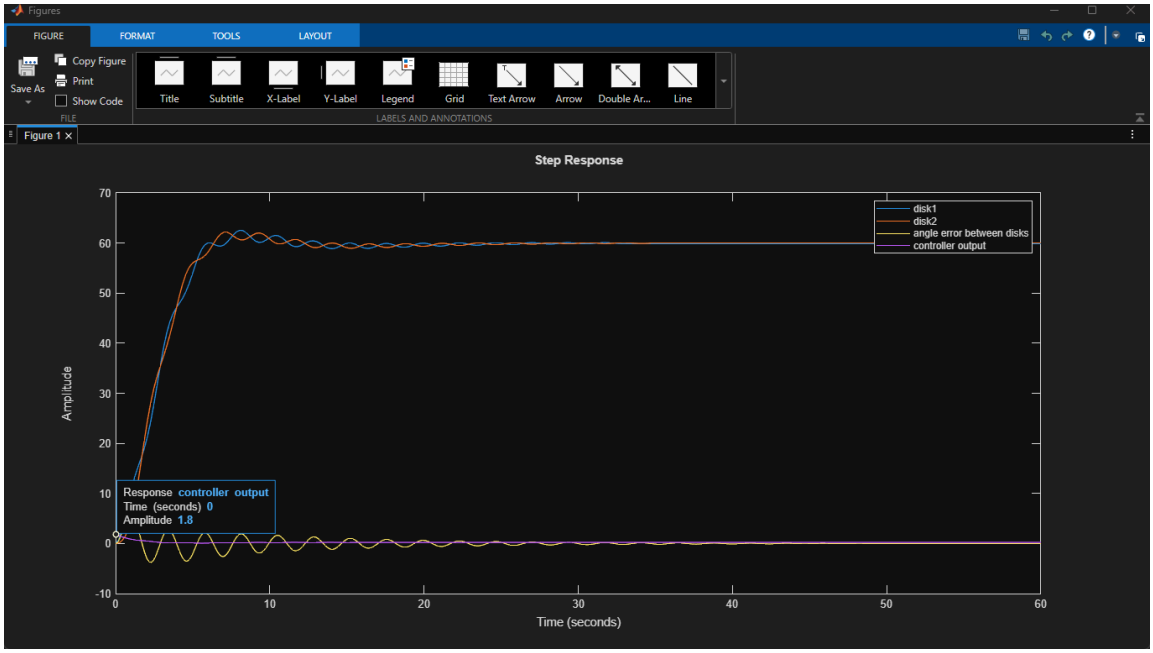
$$C(s) = \frac{s}{50}\left(\frac{1}{s+1}\right) + 0.01 + \left(\frac{.0014}{s}\right) \qquad (4)$$

Results

RiseTime: 4.3416
Transient Time: 10.7010
SettlingTime: 10.7010
SettlingMin: 55.5753
SettlingMax: 62.5438
Overshoot: 4.2396
Undershoot: 0
Peak: 62.5438
PeakTime: 8.0642

According to the stepinfo function, the transient time is under 15 seconds.





The yellow plot describing the difference between the two disks indicates that the difference between the angle of each disk remains under 10 degrees. The purple plot with the controller output indicates that the peak of the controller output is 1.8 volts, which is well under the 5 volt constraint.

Code

This section of MATLAB code was meant to generate the Bode and root locus plots of the open-loop transfer functions. It was also used to see how Bode and root locus were affected by different types of controllers. The margin command was used to make the bode plots because the command automatically computes gain and phase margin.

```
numerator = [1 0];
denominator = [1];
s = tf(numerator,denominator);
theta1 = ((21.903*s*s)+(1.3465*s)+84.2)/((s*s*s*s)+(.512*s*s*s)+(6.937*s*s)+(2.44*s)+.318)
%theta2 = ((3.588*s*s)-(1.186*s)+84.2)/((s*s*s*s)+(.512*s*s*s)+(6.937*s*s)+(2.44*s)+.318)
%gs = .1*(1/(2.68^2))*((s*s)+(s*.1)+(2.68^2));%(1+((1/.2)*s));
%gs = .005*((s*100)+1)/(s*100);
%gs = .005*(s+(1/10))/(s+(1/200));
gs = .005*(((s*100)+1)/(s*100))*((s)+1);
%gs = (s/5)+.101+(.001/s);
%gs = .005*((s+(1/10))/(s+(1/200)))*(((s*3)+1)/(s+1));
%gs = 1.7*((s*.5)+1);
%gs = .005;
%gs = ((s*.001)+1);
%a = 1
%b = a/10;
%gs = ((s*a)+1)/((s*b)+1)*.1838
%rlocus(theta1*gs)
%[K,POLES] = rlocfind(theta1*gs)
%rlocus(theta2*gs)
margin(theta1*gs)
hold on
%margin(theta2)
```

The code below is meant to put the controller and transfer functions into a closed-loop system. Stepplots are used to generate and measure the transient response of the step with a step size of 60 degrees. Additional step plots generate the output of disk θ_2, as well as plots of controller output and the angle difference between disks θ_1 and θ_2.

```
numerator = [1 0];
denominator = [1];
```

```
s = tf(numerator,denominator);
theta1 = ((21.903*s*s)+(1.3465*s)+84.2)/((s*s*s*s)+(.512*s*s*s)+(6.937*s*s)+(2.44*s)+.318)
theta2 = ((3.588*s*s)-(1.186*s)+84.2)/((s*s*s*s)+(.512*s*s*s)+(6.937*s*s)+(2.44*s)+.318)
%gs = .05*(((s*100)+1)/(s*100));
%gs = .005*(s+(1/10))/(s+(1/200));
%gs = .02*((s+(1/10))/(s+(1/200)))*(((s*3)+1)/(s+1));
%gs = .1*(((s*100)+1)/(s*100))*((s)+1);
%gs = (s/10) + .101 + (.001/s)
gs = (s/50)*(1/(s+1))+.01+(.0014/s)
%gs = 5*((s*.5)+1);
%gs = .1;
%a = 1;
%gs = .005;
%b = a/10;
%gs = (((s*a)+1)/((s*b)+1))*.2917;
system = (gs*theta1)/(1+(gs*theta1));
controller_output = ((gs)/(1+(gs*theta1)));
systemdisk2 = controller_output*theta2;
stepplot(60*system)
stepinfo(60*system)
hold on
stepplot(60*systemdisk2)
hold on
stepplot(60*(system - systemdisk2))
hold on
stepplot(60*controller_output)
legend('disk1','disk2','angle error between disks','controller output')
```

Conclusion

Using the PI and PD compensation techniques in class, we were able to get the system to have the desired transient response. Solving this complex transfer function problem has also provided experience in refining PID coefficients by hand based on general intuition of what they do. The impression we had as a group is that making controllers for sophisticated transfer functions with several poles and zeroes may require both calculation and hand refinement.